

共享单车使用量预测

任务一：线性回归

目录结构

- `sharing_bike.py`: 预测单车日租用量与温度的线性回归模型
- `day.csv`: 数据源

代码结构

1. 调用API

```
import torch
import torch.nn as nn
import pandas as pd
import matplotlib.pyplot as plt
```

2. 数据加载

```
data = pd.read_csv('day.csv')
X_df = data[['temp']]
y_df = data[['cnt']]
```

- 数据源：从`day.csv`文件中读取数据
- 特征选择：使用温度('temp')作为输入特征
- 目标变量：自行车日租用量('cnt')作为预测目标

3. 数据转换

```
X_tensor = torch.tensor(X_df.values, dtype=torch.float32)
y_tensor = torch.tensor(y_df.values, dtype=torch.float32)
```

- 将Pandas DataFrame转换为PyTorch张量，并指定float32数据类型

4. 数据可视化

```
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.scatter(X_tensor.numpy(), y_tensor.numpy(), alpha=0.5, color='b', s=20)
plt.xlabel('温度 ')
plt.ylabel('自行车日租用量')
plt.title('温度与自行车日租用量的关系')
```

```
plt.grid(True, alpha=0.3)
plt.show()
```

- 中文字体支持
- 生成散点图（设置x, y轴，标题，网格线）

5. 定义模型与前向传播

```
class LinearRegressionModel(nn.Module):
    def __init__(self):
        super(LinearRegressionModel, self).__init__()
        self.linear = nn.Linear(in_features=1, out_features=1)

    def forward(self, x):
        return self.linear(x)
```

- 继承nn.Module基类创建自定义模型
- 使用单层线性层(nn.Linear)

6. 训练配置

```
model = LinearRegressionModel()
criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
epochs = 1000
```

- 损失函数：均方误差(MSELoss)，适用于回归问题
- 优化器：随机梯度下降(SGD)，学习率设为0.01
- 训练轮数：1000个epoch

7. 训练循环

```
for epoch in range(epochs):
    predictions = model(X_tensor)
    loss = criterion(predictions, y_tensor)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

- 前向传播、损失计算、反向传播和参数更新
- 每100个epoch打印一次损失值并记录所有损失值

8. 结果及其可视化

```
weight = model.linear.weight.item()
bias = model.linear.bias.item()
print(f"\n结果:  $y = \{weight:.4f\} * x + \{bias:.4f\}$ ")

plt.figure(figsize=[15, 5])
plt.subplot(1, 2, 1)
plt.scatter(X_tensor.numpy(), y_tensor.numpy(), alpha=0.4, color='b', label='原始数据')

x_range = torch.linspace(X_tensor.min(), X_tensor.max(), 100)
y_pred = model(x_range.unsqueeze(1)).detach().numpy()
plt.plot(x_range.numpy(), y_pred, color='r', linewidth=2,
         label=f'拟合直线:  $y = \{weight:.2f\}x + \{bias:.2f\}$ ')

plt.xlabel('温度')
plt.ylabel('自行车日租量')
plt.title('线性回归拟合')
plt.legend(loc='best')
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(range(epochs), losses, color='b')
plt.xlabel('训练数据循环次数')
plt.ylabel('损失')
plt.title('训练损失曲线')
plt.grid(True, alpha=0.3)

plt.tight_layout()

plt.show()
```

- 输出权重，偏置
- 输出线性回归拟合曲线，训练损失图线

预期输出

- 数据形状信息
- 训练过程中的损失变化
- 线性回归方程
- 可视化图表：拟合效果图和损失曲线图